

Devoir surveillé

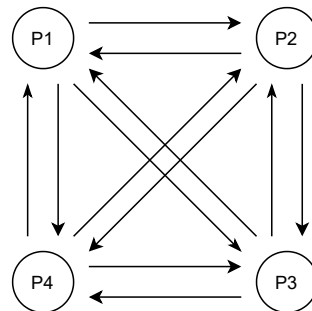
Durée 1h30
(Corrections)

Exercice 1 (*Enter the Matrix* (les tubes) - 8 pts)

Nous souhaitons réaliser une application qui crée n fils communiquant entre eux via des tubes anonymes. Nous proposons de créer une paire de tubes pour chaque couple de fils.

1°) Avec n fils, combien de tubes sont nécessaires au total ? Combien de descripteurs de fichiers sont nécessaires pour les tubes pour chaque fils ?

Solution : (1pt) Chaque fils a besoin de $n - 1$ tubes pour écrire vers les autres fils et de $n - 1$ tubes pour lire depuis les autres fils. Donc $(n - 1) \times n = n^2 - n$ tubes. Chaque fils a besoin d'un descripteur par tube donc de $2 \times (n - 1)$.



2°) Pourquoi est-il préférable d'utiliser une paire de tubes entre chaque paire de fils au lieu d'un seul ?

Solution : (1pt) La lecture n'étant pas atomique, cela évite de gérer les problèmes de concurrence entre les écritures. Si plusieurs processus écrivent dans le même tube, leurs données peuvent être «mélangées» si la taille est supérieure à `PIPE_BUF`. De plus, si nous utilisons un seul tube, le risque est qu'un fils lise son propre message.

3°) Que se passe-t-il si le nombre de fils devient trop important ?

Solution : (1pt) Le nombre de descripteurs de fichier étant limité pour chaque processus et pour le système, un nombre important de fils risque d'entraîner des erreurs lors de la création des tubes. Au départ, le père a besoin de $(n - 1) \times (n - 1) \times 2$ descripteurs, même si une grosse partie sera fermée dans la suite.

4°) Comment un fils peut-il écouter dans l'ensemble de ses tubes ? Proposez une solution en précisant les appels systèmes nécessaires.

Solution : (1pt) Il est possible de rendre la lecture dans un tube non bloquante en utilisant `fcntl`. Ainsi, le fils écoute sur chaque descripteur en lecture jusqu'à avoir des données à lire. S'il n'y a pas de donnée, `read` retourne 0. Pour éviter l'attente active, il est possible d'utiliser la surveillance de descripteurs qui a été abordée dans le dernier cours (ce qui n'était pas attendu ici).

5°) L'ensemble des tubes est représenté par une matrice de $n \times n$ où chaque case est un tableau de 2 cases. Si on considère que les processus sont numérotés de 0 à $n - 1$, pour le fils i , la ligne i contient les tubes en écriture vers les autres fils et la colonne i contient les tubes en lecture depuis les autres fils. La diagonale n'est pas utilisée.

Donnez la portion de code permettant de créer les tubes et les fils (n est spécifié comme une constante). Les descripteurs inutiles doivent être fermés.

Solution : (3pts)

```

int matrice[N][N][2];
int i, j, k;
pid_t n;

/* Création des tubes et initialisation de la matrice */
for(i = 0; i < N; i++)
    for(j = 0; j < N; j++)
        if(i != j)
            pipe(matrice[i][j]);

/* Création des fils */
for(k = 0; k < N; k++) {
    if(fork() == 0) {
        /* Je suis dans le fils */

        /* Fermeture des descripteurs inutiles */
        for(i = 0; i < N; i++) {
            for(j = 0; j < N; j++) {
                if(i == k) {
                    /* Ligne k donc tubes en écriture */
                    if(j != k)
                        close(matrice[i][j][LECTURE]);
                }
                else {
                    if(j == k) {
                        close(matrice[i][j][ECRITURE]);
                    }
                    else {
                        if(i != j) {
                            close(matrice[i][j][LECTURE]);
                            close(matrice[i][j][ECRITURE]);
                        }
                    }
                }
            }
        }
        ...
        exit(EXIT_SUCCESS);
    }
}

/* Je suis dans le père */
for(i = 0; i < N; i++) {
    for(j = 0; j < N; j++) {
        if(i != j) {
            close(matrice[i][j][LECTURE]);
            close(matrice[j][i][ECRITURE]);
        }
    }
}

```

6°) Nous souhaitons maintenant laisser la possibilité au père d'écrire des messages à ses fils en utilisant les tubes déjà créés. Expliquez les problématiques et proposez une solution pour les limiter autant que possible, sans créer de nouvelles ressources systèmes.

Solution : (1pt) Pour éviter les problèmes de concurrence, il faut s'assurer que l'écriture des données dans les tubes soit atomique. Autrement dit, il suffit de limiter la taille des messages (à 4096ko, par exemple, ce qui correspond à `PIPE_BUF`) et que les messages soient écrits avec un seul appel à `write`. Il peut également être nécessaire d'avoir à spécifier l'émetteur de chaque message (pour distinguer ceux envoyés par le père et des autres fils).

Exercice 2 (Éditeur - 8 pts)

Nous souhaitons développer un éditeur hexadécimal qui permet de visualiser ou de modifier le contenu d'un fichier. L'ensemble des commandes et leurs paramètres est passé en arguments : modification de la valeur d'un octet, affichage d'un bloc du fichier, insertion ou suppression d'un octet, insertion d'un bloc depuis un autre fichier. Vous devez faire en sorte de gérer efficacement les fichiers de grande taille en ne plaçant en mémoire qu'au maximum `MAX` octets (nous supposons que `MAX` est une constante).

1°) Donnez la portion de code permettant d'ouvrir le fichier dont le nom est passé comme premier argument du programme. Faites la gestion d'erreur et précisez à l'utilisateur lorsque le fichier n'existe pas.

Solution : (1,5 pt)

```
int fd;

fd = open(argv[1], O_RDWR);
if(fd == -1) {
    if(errno == ENOENT) {
        fprintf(stderr, "Le_fichier_%s_n'existe_pas.\n", argv[1]);
    }
    else {
        fprintf(stderr, "Erreur_lors_de_l'ouverture_du_fichier_%s_", argv[1]);
        perror("");
    }
    exit(EXIT_FAILURE);
}
```

2°) Nous souhaitons afficher un bloc du fichier. Donnez les arguments nécessaires à passer au programme, puis expliquez (sans donner le code) les différentes actions à réaliser en précisant les appels systèmes nécessaires.

Solution : (1 pt) Pour les arguments, en plus du nom du fichier et de l'option, nous avons besoin de la position de départ, ainsi que du nombre d'octets à lire (la taille du buffer). On crée un buffer de la taille demandée (attention à la taille). On se place à la position choisie avec `lseek` (option `SEEK_SET`). Ensuite, on lit le buffer avec `read`. Enfin, on affiche le contenu à l'écran. Ceci n'est pas précisé, mais nous aurons besoin également de `open` pour ouvrir le fichier et `close` pour le fermer.

3°) Que se passe-t-il si la position demandée est au-delà de la taille du fichier ? Et si le fichier contient moins d'octets que demandés ?

Solution : (1 pt) `open` retourne le nombre d'octets lus. Si la position de départ est au-delà de la fin du fichier, `open` retourne 0 (et non une erreur). De même, s'il n'y a que 100 octets à lire alors que l'utilisateur demande 200 octets, `open` retourne la valeur 100. Il faut savoir également que `lseek` permet de se positionner au-delà de la taille du fichier. Si une écriture est alors effectuée, des 0 seront ajoutés entre l'ancienne fin et les données ajoutées.

4°) Nous souhaitons supprimer un octet à une position donnée dans le fichier. Écrivez la procédure `supprimer` qui prend en paramètre `fd` le descripteur du fichier et `pos` la position de l'octet à supprimer.

Solution : (2,5 pts)

```
void supprimer(int fd, offset_t pos) {
    char buffer[MAX];
    int n;
```

```

do {
    /* Je me place à la position et je lis un bloc de données */
    lseek(fd, pos + 1, SEEK_SET);
    n = read(fd, buffer, MAX);
    if(n > 0) {
        /* Des données ont été lues, je les réécris 1 octet avant */
        lseek(fd, pos, SEEK_SET);
        write(fd, buffer, n);
        pos += n;
    }
} while(n == MAX);
/* Je supprime le dernier octet du fichier */
ftruncate(fd, pos - 1);
}

```

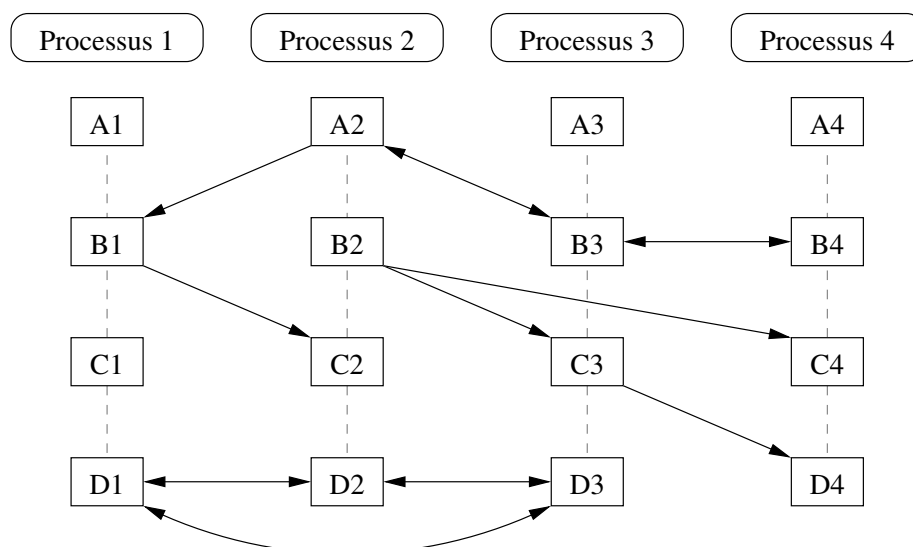
5°) Enfin, nous souhaitons permettre l'insertion d'octets d'un second fichier dans le fichier à une position donnée. Précisez les arguments nécessaires à passer au programme. Expliquez comment procéder en précisant les appels systèmes utilisés. Nous supposons que la taille des données à insérer est inférieure à MAX.

Solution : (2 pt) Nous avons besoin des noms des deux fichiers, de l'option, de la position où insérer les données, la position et la taille des données à copier.

La première étape consiste à décaler les données pour permettre d'insérer les données. Pour cela, on doit déplacer les données bloc par bloc de *taille* octets, en partant de la fin du fichier. On se place à la position *SEEK_END - MAX* avec *lseek*, on lit avec *read*, on se déplace avec *lseek* et on écrit les données avec *write* et ainsi de suite. Une fois les données déplacées, on se place dans le second fichier avec *lseek*. On lit les données avec *read*. Puis on se place à la position où insérer les données avec *lseek* et on écrit les données avec *write*.

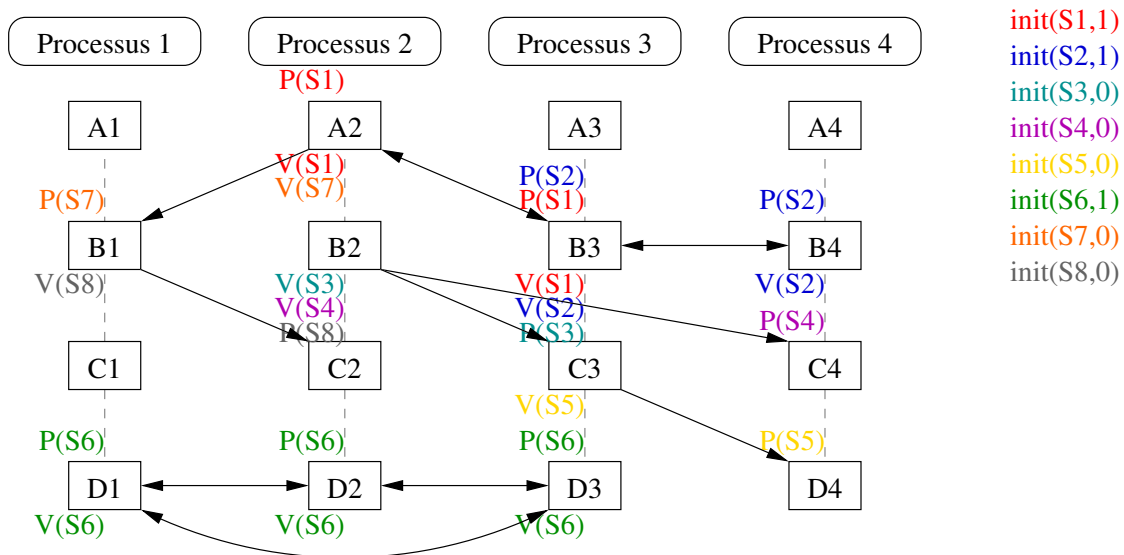
Exercice 3 (Sémaphores de Dijkstra - 4 pts)

Soit le diagramme de précédence suivant, représentant 4 processus contenant chacun 4 blocs s'exécutant les uns après les autres :



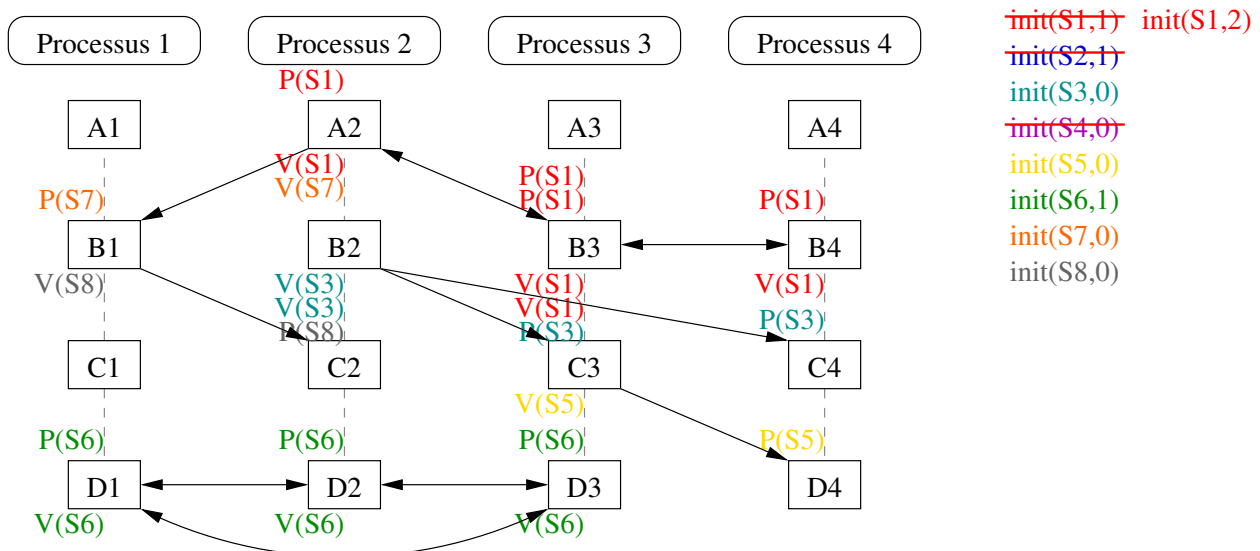
1°) Proposer une solution à base de sémaphores pour résoudre ces contraintes (1 sémaphore pour chaque cas).

Solution : (2pts) Voici une solution :



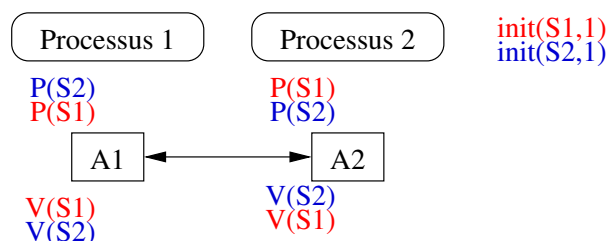
2°) Proposez une solution pour réduire le nombre de sémaphores.

Solution : (1pts) Le sémaphore 4 peut être supprimé en réutilisant le sémaphore 3. Le sémaphore 2 peut être supprimé, le sémaphore 1 peut être réutilisé avec une initialisation à 2.



3°) Donnez un diagramme de précedence qui peut induire un interblocage en fonction de l'exécution. Expliquez différentes exécutions possibles.

Solution : (1pts) Soit le diagramme suivant :



Si le processus 1 exécute $P(S2)$ puis que le processus 2 exécute $P(S1)$, alors les processus attendront infiniment sur la seconde opération. Par contre, si le processus 1 exécute $P(S2)$ et $P(S1)$ avant que le processus 2 n'exécute $P(S1)$, alors il n'y aura pas d'interblocage.